

플로우램프 최종보고서

(Flow Lamp)

본 보고서를 2026년도 Project [플로우램프]의
최종보고서로 제출합니다.

학부(전공)/학과: 컴퓨터공학과

담당(지도)교수: 임환섭

팀명: 디즈니

제출일: 2026년 06월 11일

팀 장 : 권태정 (20212909)

팀원1 : 이준영 (20212943)

팀원2 : 최영인 (20212888)

팀원3 : 서성준 (20212925)

동의대학교 컴퓨터공학과 학과장 귀하

〈 요약 문 〉

프로젝트 목표 (300자내외)	본 프로젝트의 목표는 사용자의 위치와 작업 환경을 실시간으로 인식하여 최적의 조명 방향과 밝기를 자동으로 제공하는 스마트 조명 시스템 '플로우 램프'를 구현하는 것입니다. 카메라를 통해 착석 여부와 작업 영역을 파악하고, 제스처 인식을 통해 비접촉식으로 조명을 제어합니다. 이를 통해 수동 조작의 번거로움을 없애고 작업 흐름을 방해하지 않는 사용자 맞춤형 조명 환경을 제공합니다. 또한 시간에 따른 주변 광량에 따라 사용자가 느끼는 불편함을 최소화 하도록 설정합니다. 사용자가 타이머를 설정하게 된다면 소리가 아닌 빛으로 알려주게 되며, 더 나아가서는 졸음을 감지하여 깨우고 부적절한 자세를 취한다면 경고를 보냅니다.		
내용 (500자내외)	플로우 램프는 하드웨어와 딥러닝 소프트웨어가 융합된 능동형 키네틱 로봇입니다. 3D 프린팅 프레임과 다이내믹셀 모터를 결합하고, 3축 역운동학(IK) 및 PID 제어를 적용해 부드러운 관절 모션을 구현합니다. 시스템 제어는 OpenCV와 MediaPipe 등 컴퓨터 비전 기술을 활용합니다. 특히 별도의 조도 센서 없이 카메라 픽셀 데이터만으로 주변 밝기를 분석하는 센서리스 지능형 제어를 통해 광량을 자동 최적화합니다. 또한, 앱 제어는 물론 Zero-UI 기반의 손 제스처 제어를 도입하여 직관성을 높였으며, 인간-로봇 상호작용(HRI)을 고려한 감정 상태 머신 설계를 통해 생활 밀착형 키네틱 아트로 기능하도록 개발합니다.		
기대효과 (200자내외)	최적의 맞춤형 조도와 위치를 자동으로 제공하여 사용자의 시력 저하를 예방하고 학습 집중력을 크게 향상시킵니다. 기술적으로는 단일 카메라로 조도, 제스처, 졸음 등을 모두 파악해 다중 센서 구성 대비 부품 원가를 절감할 수 있습니다. 또한 주변 밝기에 따른 자동 전력 관리로 에너지를 절약하며, 누구나 쉽게 사용할 수 있어 디지털 격차 해소에도 기여합니다.		
Keywords	스마트 조명	제스처 인식	키네틱 로봇
	센서리스 제어	컴퓨터 비전	HRI

목 차

I. 서론	1
1. 프로젝트의 배경 또는 필요성	1
2. 프로젝트의 목표	1
II. 프로젝트 수행 내용	2
1. 프로젝트의 내용	2
2. 프로젝트 추진전략 및 방법(체계)	2
III. 결과 활용 및 기대효과	5
1. 프로젝트 결과의 활용방안	5
2. 프로젝트 결과의 기대효과	5
IV. 기타	6
[참고문헌]	6

I. 서론

1. 프로젝트의 배경 또는 필요성

최근 스마트 홈 및 IoT 기술의 발전으로 조명 시스템 또한 단순한 밝기 조절을 넘어 사용자 환경에 맞춰 자동으로 동작하는 지능형 시스템으로 발전하고 있습니다. 특히 학습 환경에서 조명의 밝기와 방향은 눈의 피로도, 집중력, 작업 효율에 큰 영향을 미치는 중요한 요소입니다.

그러나 대부분의 학생들은 책상 조명의 위치와 밝기를 고려하지 않고 학습을 진행하며, 이로 인해 그림자 발생, 시야 방해, 눈의 피로 증가 등의 문제가 발생합니다. 또한 조명을 수동으로 조작해야 하는 과정은 학습의 흐름을 끊는 요소로 작용합니다.

이러한 문제를 해결하기 위해 사용자의 위치와 작업 환경을 인식하여 조명의 방향과 밝기를 자동으로 조절하고, 비접촉 방식의 제스처 인식을 통해 조명을 제어할 수 있는 스마트 조명 시스템의 필요성이 대두되고 있습니다. 본 프로젝트는 이러한 요구를 반영하여 학습 환경을 개선하고 사용자 중심의 편의성을 제공하는 '플로우 램프(Flow Lamp)'를 제안합니다.

2. 프로젝트의 목표

본 프로젝트의 목표는 사용자의 위치 및 작업 환경을 실시간으로 인식하여 조명의 방향과 밝기를 자동으로 조정하는 스마트 조명 시스템을 구현하는 것입니다. 구체적으로는 다음과 같은 기능을 목표로 합니다.

1. 사용자가 책상에 착석하면 자동으로 조명이 활성화되는 기능 구현
2. 카메라를 활용하여 사용자의 위치 및 작업 영역을 인식하고, 최적의 조명 방향과 밝기를 자동으로 조정하는 기능 구현
3. 손 제스처 인식을 통해 조명의 밝기 및 동작을 비접촉 방식으로 제어하는 인터페이스 구현
4. 학습 및 작업 흐름을 방해하지 않는 사용자 중심의 조명 환경 제공
5. 램프를 직접 조작할 수 있는 전용 어플리케이션 구현

이를 통해 기존 조명 시스템의 한계를 개선하고, 사용자 맞춤형 스마트 조명 시스템을 구현하는 것을 최종 목표로 합니다.

II. 프로젝트 수행내용

1. 프로젝트의 내용

(1) 공학적 요소 및 제한조건

합성 및 분석: OpenCV를 통한 2D 좌표 추출 및 3축 역운동학(IK) 모델링을 통한 물리적 동기화 구현

제작 및 시험: 3D 프린팅(PLA) 프레임 제작 및 PID 제어를 통한 'Flow' 모션(부드러운 가속/감속) 튜닝

원가/안정성: 80만 원 예산 내 다이내믹셀 모터 확보 및 Li-Po 배터리 보호회로 구축으로 무선 안정성 확보

미학/내구성: 배선 매립형 설계로 심미성 극대화 및 알루미늄 브라켓을 통한 관절 내구성 강화

(2) 비공학적 요소

연구 방법: 기존 IoT 기기의 불편함(앱 조작 등)을 분석하여 Zero-UI(제스처 제어) 기반의 직관적 UX 연구

HRI 설계: 인간-로봇 상호작용(Human-Robot Interaction)을 고려한 감정 상태 머신(State Machine) 설계로 정서적 유대감 형성

3) 프로젝트 설계의 독창성

능동적 키네틱 로봇: 사용자의 움직임을 실시간으로 추적하여 빛이 스스로 따라오는 능동적 시스템(수동성 탈피)

센서리스(Sensorless) 지능형 제어: 별도의 조도 센서 없이 카메라 비전만으로 주변 밝기를 파악해 광량을 자동 최적화

기술과 예술의 융합: 로봇틱스 기술을 인테리어 소품에 녹여낸 생활 밀착형 키네틱 아트

4) 기타

확장성: 향후 음성 인식 및 다중 램프 군집 제어 시스템으로의 하드웨어 확장 가능성

사회적 기여: 시각적 알림을 통한 딴짓 방지 및 집중력 향상 도우미로 활용

2. 프로젝트 추진전략 및 방법(체계)

1) 프로젝트 추진 일정 및 방법

본 프로젝트는 손동작 인식 기반 스마트 램프 시스템 구현을 목표로 하며, 이를 위해 체계적인 정보 수집과 협업, 단계별 개발 방법을 적용합니다.

정보 수집 및 시스템 설계: 컴퓨터 비전 및 딥러닝 기술 관련 자료를 조사하고, OpenCV, MediaPipe, TensorFlow와 같은 라이브러리의 활용 방법을 학습합니다. 또한, 손동작 인식 및 제스처 인터페이스 관련 논문과 사례를 참고하여 시스템 설계에 반영합니다.

역할 분담 및 협업: 영상 처리 및 인공지능 모델 개발, 하드웨어 제작, 시스템 통합 및

테스트 등으로 역할을 나누고, 정기적인 회의를 통해 진행 상황을 공유합니다. 협업 도구를 활용하여 코드 및 데이터를 공동 관리하며, 문제 발생 시 팀원 간 토의를 통해 해결 방안을 도출합니다.

단계별 개발: 데이터 수집 및 전처리를 수행한 후 제스처 인식 모델을 설계하고 학습 시킵니다. 이후 OpenCV 기반의 실시간 영상 처리 시스템과 학습된 모델을 결합하여 기능을 구현합니다. 마지막으로, 아두이노 또는 라즈베리파이를 활용하여 램프 하드웨어와 연동하고, 전체 시스템을 통합하여 테스트를 진행합니다.

문제 해결 및 성능 개선: 다양한 환경에서의 테스트를 통해 인식률 저하 문제를 분석하고, 데이터 보강 및 모델 튜닝을 통해 성능을 개선합니다. 하드웨어 동작 오류 발생 시 회로 점검 및 코드 디버깅을 통해 안정성을 확보합니다.

3. 기능/비기능 요구사항

요구사항 ID	유형	요구사항 명칭	우선순위	상세 설명	수용 기준
SFR-001	CORE	사용자 감지	Must	센서를 이용하여 사용자가 근처에 접근하는 것을 감지하여 카메라의 전원을 켜다.	• 사용자 1m 이내 접근 시 감지 • 감지 후 1초 이내 카메라 전원 ON • 감지 정확도 90% 이상
SFR-002	CORE	핑거스냅 전원 온	Must	사용자가 핑거스냅을 하면 카메라를 이용해 감지하고 램프의 전원을 켜다.	• 핑거스냅 인식 정확도 90% 이상 • 1초 이내 전원 ON
SFR-003	CORE	자동 밝기 조절	Must	램프의 전원이 켜지면 주위의 밝기를 인식하고 사용자의 눈에 가장 편안한 밝기를 자동으로 설정한다.	• 주변 조도에 따라 밝기 자동 조절 • 목표 조도 오차 $\pm 10\%$ 이내 • 3초 이내 조절 완료
SFR-004	CORE	자동 위치 조절	Must	램프의 전원이 켜지면 카메라로 작업 공간을 인식 한 후 적절한 위치에 램프를 위치 시킨다.	• 작업 영역 인식 정확도 80% 이상 • 램프 위치 오차 $\pm 5\text{cm}$ 이내
SFR-005	CORE	핑거스냅 전원 오프	Must	사용자가 핑거스냅을 하면 카메라를 이용해 감지하고 램프의 전원을 끈다.	• 핑거스냅 인식 정확도 90% 이상 • 제스처 입력 후 1초 이내 전원 OFF • 오인식률 5% 이하
SFR-006	CORE	자동 전원 오프	Must	사용자가 작업공간을 10분 이상 비우거나 램프의 기능을 켜두지 않으면 자동으로 전원을 끈다.	• 10분 이상 사용자 미감지 시 전원 OFF • 오동작률 5% 이하
SFR-007	UX	제스처 밝기 조절	Must	사용자가 손 제스처를 통해 램프 밝기를 조절할 수 있다.	• 제스처 인식 정확도 85% 이상 • 제스처 입력 후 1초 이내 밝기 반영 • 밝기 조절 범위 0~100% 지원
SFR-008	UX	제스처 위치 조절	Must	사용자가 손 제스처를 통해 램프의 위치를 조절할 수 있다.	• 제스처 인식 정확도 85% 이상 • 제스처 입력 후 1초 이내 위치 반영 • 위치 오차 $\pm 5\text{cm}$ 이내
SFR-009	UX	타이머	Won't	사용자가 램프 사용 시간을 설정할 수 있는 타이머 기능을 제공한다.	• 1분 단위 시간 설정 가능 • 설정 오차 $\pm 1\text{초}$ 이내 • 설정 후 즉시 타이머 동작 시작
SFR-010	UX	타이머 알림	Must	설정된 시간이 종료되면 사용자에게 알림을 제공한다.	• 설정 시간 종료 후 1초 이내 알림 발생 • 알림 지속 시간 3초 이상 • 알림 인지율 90% 이상
SFR-011	LINK	앱 전원 온	Must	모바일 앱을 통해 램프 전원을 원격으로 켤 수 있다.	• 앱 명령 후 2초 이내 전원 ON • 명령 성공률 95% 이상 • 네트워크 연결 상태에서 정상 동작
SFR-012	LINK	앱 전원 오프	Must	모바일 앱을 통해 램프 전원을 원격으로 끌 수 있다.	• 앱 명령 후 2초 이내 전원 OFF • 명령 성공률 95% 이상 • 네트워크 연결 상태에서 정상 동작
SFR-013	LINK	앱 위치 조절	Must	모바일 앱을 통해 램프의 위치를 원격으로 조절할 수 있다.	• 앱 명령 후 2초 이내 위치 반영 • 위치 조절 성공률 95% 이상 • 위치 오차 $\pm 5\text{cm}$ 이내

SFR-014	LINK	앱 밝기 조절	Must	모바일 앱을 통해 램프의 밝기를 원격으로 조절할 수 있다.	• 앱 명령 후 2초 이내 밝기 반영 • 밝기 조절 범위 0~100% 지원 • 설정 값과 실제 밝기 오차 $\pm 10\%$ 이내 • 명령 성공률 95% 이상
SFR-015	LINK	앱 타이머 설정	Must	모바일 앱을 통해 램프의 사용 시간을 원격으로 설정할 수 있다.	• 앱 명령 후 2초 이내 타이머 설정 반영 • 1분 단위 시간 설정 가능 • 설정 시간 오차 ± 1초 이내 • 명령 성공률 95% 이상
SFR-016	REAL	졸음 감지	Should	카메라를 통해 사용자의 눈 깜빡임 및 얼굴 상태를 분석하여 졸음을 감지한다.	• 졸음 감지 정확도 85% 이상 • 졸음 상태 3초 이상 지속 시 감지 • 오탐률 10% 이하
SFR-017	REAL	졸음 경고	Should	졸음이 감지되면 사용자에게 알림(소리 또는 조명)을 제공한다.	• 졸음 감지 후 1초 이내 경고 발생 • 경고 지속 시간 3초 이상 • 사용자 인지율 90% 이상
SFR-018	CORE	자세교정	Could	카메라를 통해 사용자의 자세를 분석하여 올바르지 않은 자세를 감지한다.	• 자세 인식 정확도 85% 이상 • 잘못된 자세 5초 이상 지속 시 감지 • 오탐률 10% 이하

SFR-019	LINK	Must	라즈베리파이에서 넘겨준 데이터를 앱으로 시각화 하여 보여준다.	1개 이상의 데이터가 입력되어야 그래프 시각화
---------	------	------	------------------------------------	---------------------------

4. 프로젝트 일정

[표 1] 프로젝트 추진 일정

No	프로젝트 활동 내용	추진 일정															기간 (주)	추진 방법
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		
1	관련 정보 수집																	인터넷 자료조사, 기술 조사
2	시스템 설계																	전체 구조도, 개발환경 구축
3	데이터 수집 및 학습																	데이터 전처리, 모델 설계 및 학습
4	구현 및 개발																	기능 구현, AI 하드웨어 통합
5	테스트 및 개선																	인식률, 속도 개선, 시스템 안정화
6	문서화 및 발표준비																	보고서 작성 및 피피티 제작

2) 프로젝트 추진 체계

[표 2] 프로젝트 추진 체계

성명 / 학번		수행내용	역할
사진	권태정	- PM - 하드웨어 조립 및 핸드트래킹 파이프라인 구축	팀장
	20212909		
사진	이준영	- 다관절 모터 통신 모듈 및 제어부 구축 - 전원공급 회로 및 LED조명 시스템 설계	팀원
	20212943		
사진	최영인	- 전체적인 하드웨어 설계 - 사용자 제어용 UI/UX 어플리케이션 개발	팀원
	20212888		
사진	서성준	- 임베디드로컬 서버 및 통신 아키텍처 구축 - 제스처 매핑 및 이벤트 처리 로직 구현	팀원
	20212925		

프로젝트명		플로우 램프 (Flow Lamp)			프로젝트 관리자		권태정	
wbs번호	작업이름	선행작업	시작날짜	완료날짜	산출물	재사용여부	자원이름	
1	프로젝트 계획		2026-03-02	2026-03-27	프로젝트 계획서			
1.1	팀 구성 및 역할 분담		2026-03-02	2026-03-06	팀 구성도			
1.2	프로젝트 주제 및 범위 확정		2026-03-09	2026-03-13	프로젝트			
1.3	프로젝트 계획서 작성		2026-03-13	2026-03-19	프로젝트 계획서			
1.4	기술 스택 선정 및 개발 환경 구축		2026-03-19	2026-03-22	기술 스택			
1.5	전체 일정 수립 및 WBS 작성		2026-03-22	2026-03-23	WBS			
1.6	주제 발표 준비 및 발표		2026-03-23	2026-03-27	계획 발표 자료			
2	사용자 요구사항 정의		2026-03-27	2026-03-30	요구사항 명세서			
2.1	사용자 스토리 작성		2026-03-27	2026-03-27	사용자 스토리			
2.2	핵심 기능 우선순위 선정 및 확정		2026-03-28	2026-03-28	기능 우선순위			
2.3	하드웨어 제어 기능 명세 확정		2026-03-29	2026-03-29	기능 명세서			
2.4	기술 구현 가능성 검토 및 부품 대조		2026-03-30	2026-03-30	기술 검토 보고서			
2.5	주간 회의 및 결과 기록		2026-03-30	2026-03-30	주간 회의록			
3	스프린트 1 (앱 UI/UX 개발)		2026-03-30	2026-04-16	UI 프로토타입			
3.1	사용자 스토리 작성 및 우선순위 선정		2026-03-30	2026-04-02	스토리 백로그			
3.2	앱 인터페이스 설계		2026-04-02	2026-04-08	UI 설계서			
3.3	메인 화면 및 제어 기능 구현		2026-04-09	2026-04-14	UI 구현 코드			
3.4	스프린트 1 리뷰 및 회고		2026-04-15	2026-04-16	스프린트 1 회고록			
4	스프린트 2 (하드웨어 구상 및 부품확보)		2026-03-30	2026-04-16	하드웨어 설계서			
4.1	하드웨어 기능 명세 및 규격 검토		2026-03-30	2026-03-31	하드웨어 명세서			
4.2	회로도 설계 및 부품 리스트 확정		2026-04-01	2026-04-01	회로도			
4.3	부품 주문 및 수급 완료		2026-04-01	2026-04-14	실물 부품			
4.4	외관 3D 모델링 설계		2026-04-02	2026-04-10	3D 설계 파일			
4.5	외부 프레임 3D모델링 제작		2026-04-10	2026-04-14	최종 3D 모델링 본			
4.6	스프린트 2 리뷰 및 회고		2026-04-14	2026-04-16	스프린트 2 회고록			
5	스프린트 3 (하드웨어 제작 및 조립)		2026-04-16	2026-05-08	하드웨어 시제품			
5.1	회로 기판 배선 및 하우징 제작		2026-04-16	2026-04-28	조립 결과물			
5.2	부품 조립 및 기구부 구동 테스트		2026-04-29	2026-05-05	구동 테스트 영상			
5.3	기초 펌웨어 작성 및 동작 확인		2026-05-06	2026-05-08	펌웨어 소스코드			
5.4	스프린트 3 리뷰 및 회고		2026-05-08	2026-05-08	프로젝트 중간 설계서			
6	스프린트 4 (동작 학습 및 연동)		2026-05-11	2026-06-05	통합 시스템			
6.1	Flow Lamp 움직임 알고리즘 설계		2026-05-11	2026-05-20	알고리즘 설계서			
6.2	센서 데이터 기반 동작 학습 구현		2026-05-21	2026-05-29	학습 코드 및 데이터			
6.3	앱-하드웨어 간 통신연동 및 통합		2026-05-25	2026-06-02	통신 모듈 코드			
6.4	스프린트 4 리뷰 및 회고		2026-06-03	2026-06-05	스프린트 4 회고록			
7	스프린트 5 (통합 테스트 및 배포)		2026-06-08	2026-06-12	최종 시스템			
7.1	전체 시스템 통합 테스트		2026-06-08	2026-06-10	테스트 보고서			
7.2	사용자 경험(UX) 테스트 및 버그 수정		2026-06-10	2026-06-11	사용자 매뉴얼			
7.3	최종 결과물 시연 준비 및 배포		2026-06-11	2026-06-12	프로젝트 최종 설계서			
7.4	스프린트 5 리뷰 및 전체 프로젝트 회고		2026-06-11	2026-06-12	스프린트 5 및 전체 프로젝트 회고록			
8	프로젝트 완료		2026-06-08	2026-06-12	최종 결과물			
8.1	최종 결과 보고서 작성		2026-06-08	2026-06-12	프로젝트 완료 보고서			
8.2	프로젝트 발표 및 시연		2026-06-12	2026-06-12	최종 발표자료			

III. 결과 활용 및 기대효과

1. 프로젝트 결과의 활용방안

- 개인 학습 및 사무용 스마트 조명: 사용자가 직접 조절하지 않아도 카메라가 주변 밝기를 실시간으로 측정하여 최적의 조도를 유지해 주는 '어댑티브 라이팅(Adaptive Lighting)' 스탠드로 활용할 수 있습니다.

수험생 집중력 및 시력 보호 도구: 졸음 감지 기능과 더불어, 주변 환경에 맞춰 눈이 가장 편안한 밝기 값을 자동으로 설정함으로써 장시간 학습 시 시력 저하를 예방하는 도구로 활용합니다. 또한 데이터를 시각화 하여 사용자가 자세 및 학습 점수를 자체적으로 점검 할 수 있습니다.

2. 프로젝트 결과의 기대효과

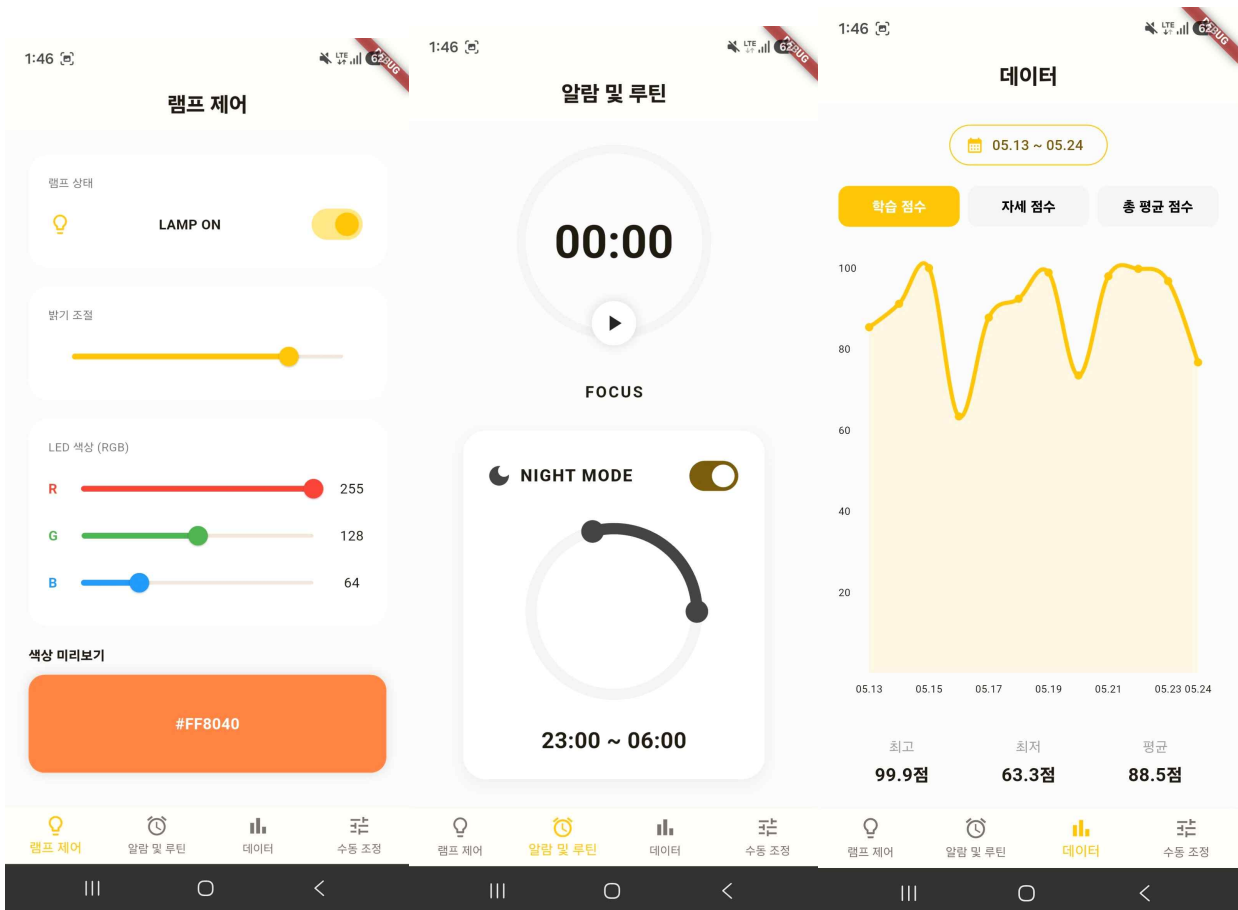
기술적 측면	카메라 기반 휘도 분석 기술 확보: 별도의 하드웨어 조도 센서 없이 카메라 이미지의 픽셀 데이터를 분석하여 주변 밝기를 추정하고 제어하는 소프트웨어 알고리즘 구현 역량을 강화합니다. 지능형 자동 제어 시스템 구축: 사용자 조작(수동)과 환경 반응 간의 우선순위를 결정하는 피드백 제어 로직을 통해 안정적인 임베디드 시스템 설계 경험을 확보합니다.
경제적 측면	부품 간소화 및 원가 절감: 카메라 하나로 제스처, 졸음 감지, 조도 측정을 모두 수행함으로써 다수의 센서 사용을 줄여 제조 단가를 낮춘 고효율 제품 구현이 가능합니다. 스마트 에너지 관리: 주변이 충분히 밝을 때는 전력 소모를 최소화하도록 자동 밝기 조절이 이루어져 전기 요금 절감 및 제품 수명 연장 효과를 기대할 수 있습니다.
사회적 측면	사용자 시력 건강 증진: 부적절한 조명 밝기로 인한 안구 건조 및 시력 감퇴를 방지하여 현대인의 눈 건강 관리 플랫폼으로서 기여합니다. 디지털 격차 해소: 복잡한 기기 조작에 서툰 사용자도 전원만 켜면 기기가 알아서 최적의 환경을 맞춰주는 기술을 통해 누구나 쉽게 기술의 혜택을 누릴 수 있습니다.

IV. 프로젝트 최종 수행 결과

앱개발 현황

사용자 편의성을 최우선으로 인터페이스 설계를 완료했습니다.

학습타이머기능, 야간 블루라이트 모드, 밝기 색상 조절, 램프 각도 제어, 데이터 시각화 기능구현 완료



핵심 기술 구현

핸드 트래킹 기술

21개 랜드마크 추출을 통한 비 접촉 로직 구현 완료

MediaPipe와 OpenCV를 통합하여 저사양환경에서도 끊김 없는 30FPS 이상의 분석 속도를 확보

8:08 N



수동 제어



1번과 4번 모터를 버튼을 누르는 동안 수동 제어합니다.



1번 모터



4번 모터



램프 제어



알람 및 루틴

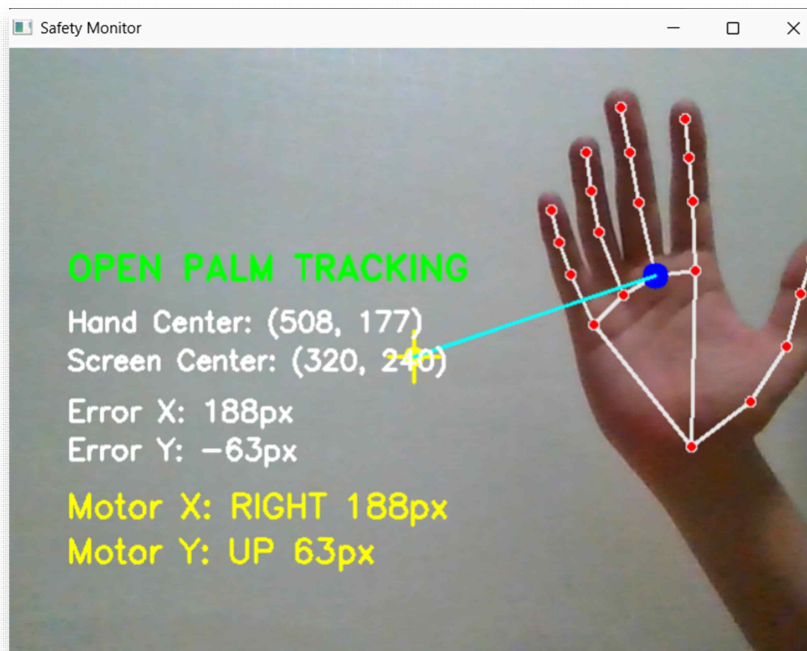


데이터



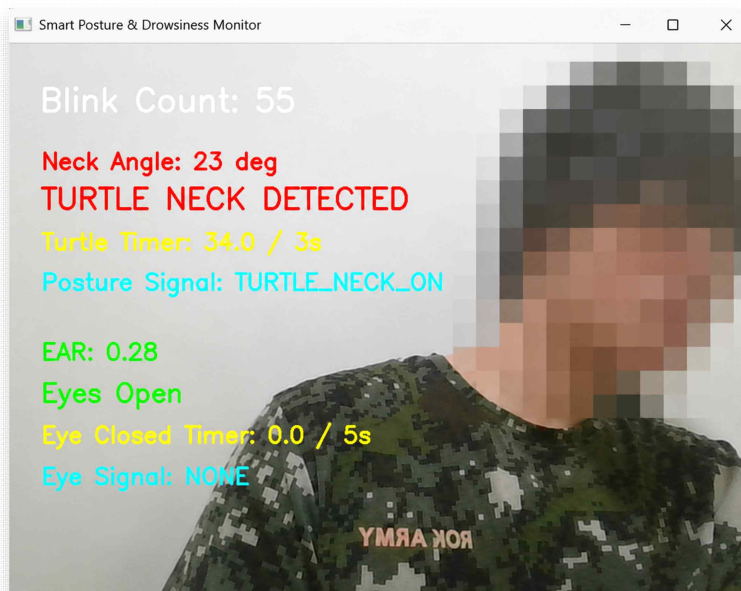
수동 제어





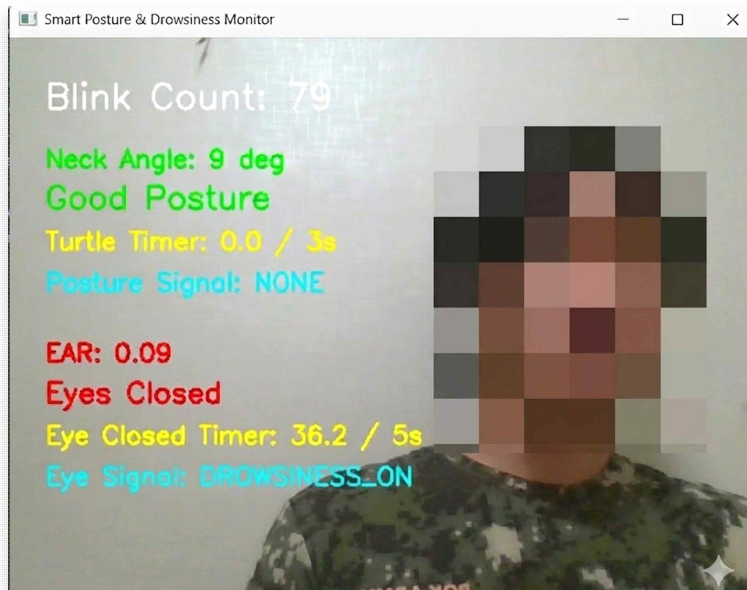
거북목 감지

Pose Estimation을 이용한 경추각도 계산 및 경고 시스템 완성



졸음(깜빡임) 감지

EAR(Eye Aspect Ratio) 기반 실시간 졸음 감지 코드 최적화

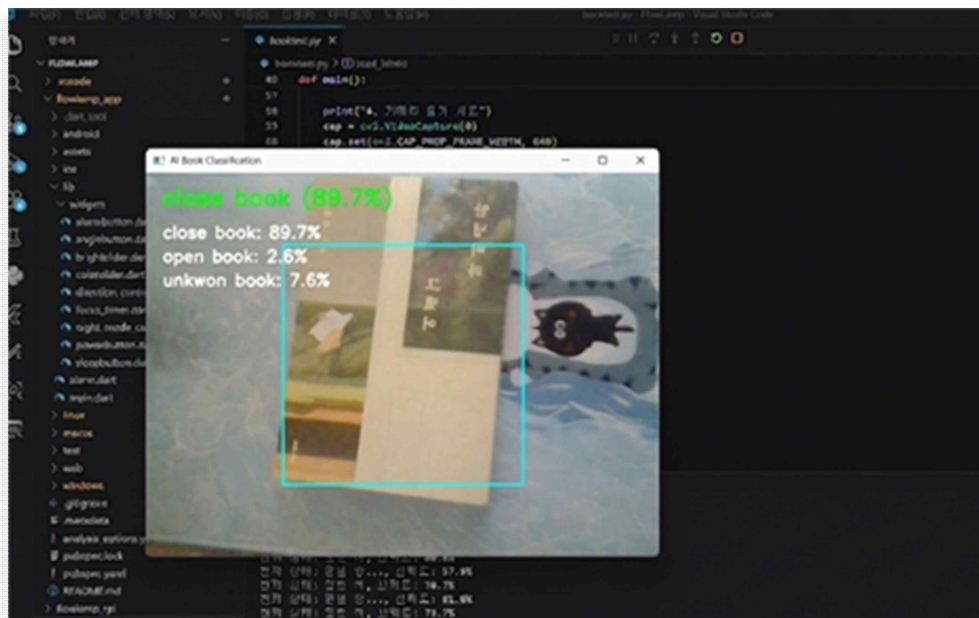


AI모델 학습 : Teachable Machine 활용

데이터셋 구축 : Google Teachable Machine을 활용하여 실제 책상 환경의 이미지 데이터500+장 학습

클래스 분류: Open Book(공부 중), Close Book(휴식) Empty(자리비움) 등상황별 학습 모델 생성

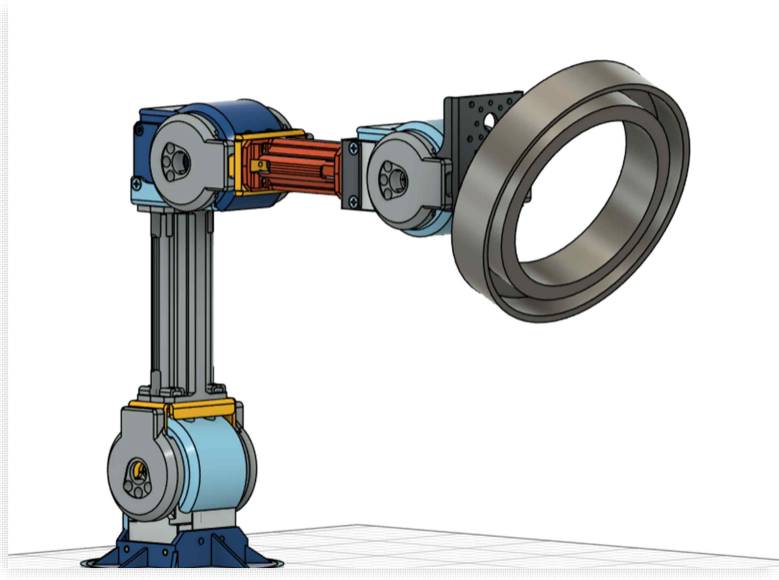
활용 방안:단순 타이머가 아닌책이 감지될 때만 학습 시간으로 인정하는 지능형 로직 적용



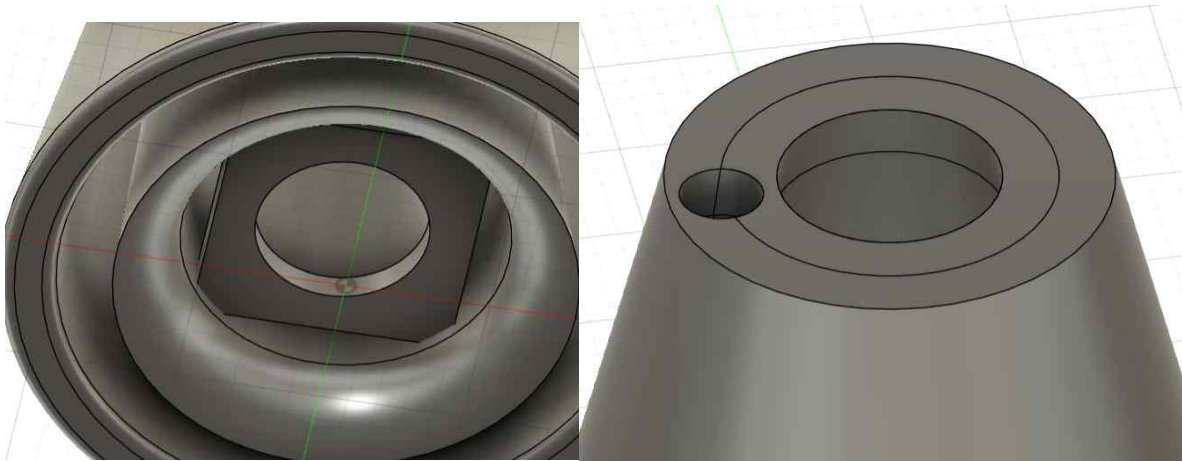
3D 모델링 및 하드웨어 기획

다관절로봇 암 구조 설계

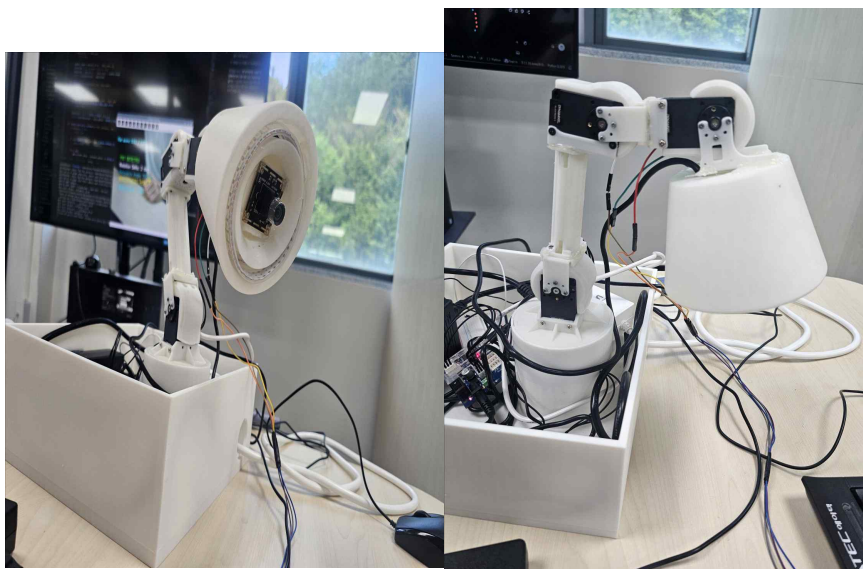
동적 각도 제어 : 거북 목 감지 시 시선 유도를 위한 램프 헤드(링) 각도 조절
기구 설계 : 안정적인 모터 토크와 회전 반경을 고려한 다관절암 모델링



램프 헤드 초기 설계



램프 헤드 최종 설계



3D 모델링 설계에 따른 하드웨어 최종 완성 사진

3.2 역할별 상세 수행 내용

권태정 (팀장): 프로젝트 관리(PM), 하드웨어 조립 및 핸드트래킹 파이프라인 구축 진행

이준영: 다관절 모터 통신 모듈 구축 및 전원공급/LED 시스템 설계 완료

최영인: 3D 프린팅 기반 프레임 설계 및 앱 UI/UX 어플리케이션 개발 진행

서성준: 임베디드 로컬 서버 아키텍처 설계 및 제스처 매핑 로직 구현 중

설계 목표의 중요도 및 달성도

목표	중요도(%)	달성도(%)	수행내용
UI/UX	10	10	전용 어플리케이션 인터페이스 설계 및 메인 제어 화면 구현 완료
제스처인식	10	10	MediaPipe 기반 손가락 랜드마크 추출 및 핑거 포인팅 제스처 매핑 로직 설계 완료
자세/상태	20	15	OpenCV 및 카메라 비전을 활용한 사용자 착석 여부 감지 및 줄음 분석 알고리즘 기초 설계 완료
하드웨어 제어	40	40	다이나믹셀 모터 제어 보드 구축, 전원 공급 회로 및 LED 시스템 설계 및 부품 조립 완료
Flow 기능	20	20	능동적 움직임 알고리즘 설계 중
합계	100	95	전체적인 시스템 아키텍처 수립 및 핵심 하드웨어 구성 완료, 지능형 동작 연동 단계 진입 중

V. 결론 및 향후 과제

1. 프로젝트 수행 결과 요약

본 프로젝트는 컴퓨터 비전 AI와 하드웨어 제어 기술, 그리고 모바일 애플리케이션을 결합하여 사용자 맞춤형 학습 환경을 제공하는 스마트 조명 시스템 ‘플로우램프(Flow Lamp)’를 성공적으로 구현하였습니다. 기존의 수동적인 스탠드 조명의 한계를 벗어나, 카메라 비전 데이터를 바탕으로 사용자의 행동(자세, 졸음, 제스처 등)을 실시간으로 분석하고 이를 모터 구동과 LED 피드백, 그리고 모바일 앱의 데이터 시각화로 연결하는 통합 IoT 시스템 아키텍처를 완성하였습니다.

2. 핵심 구현 성과

소프트웨어 및 인공지능 측면에서는 기획했던 대부분의 핵심 기능이 목표치에 도달하여 원활하게 구동되었습니다.

비전 기반 스마트 트래킹 및 제어: 손바닥 인식(Open Palm)을 통한 램프 헤드 트래킹 기능과, 주먹을 쥐고 돌리는 제스처를 아크탄젠트(atan2)로 계산하여 직관적으로 LED 밝기를 조절하는 Zero-UI를 성공적으로 구현했습니다.

스마트 스터디 매니저 (자세 및 집중도 분석): MediaPipe FaceMesh와 Pose를 활용하여 사용자의 EAR(눈 깜빡임)을 분석해 졸음을 감지하고, 귀와 어깨의 각도를 통해 거북목 자세를 인식합니다. 거북목 감지 시 LED가 붉은색으로 점멸하여 사용자에게 즉각적인 피드백을 제공합니다.

데이터 정량화 및 모바일 앱 연동: 사용자의 착석 여부, 바른 자세 유지 시간 등을 종합하여 '학습 점수'와 '자세 점수'로 정량화합니다. 이 데이터는 SQLite DB에 저장되며, 전용 모바일 앱을 통해 날짜별 그래프로 시각화되어 사용자 스스로 학습 습관을 점검할 수 있습니다.

모바일 앱 원격 제어: 앱을 통한 1번, 4번 모터의 독립적인 상세 앵글 제어, 타이머 설정(종료 시 LED 점멸 알림), 세밀한 RGB LED 조명 제어 등 사용자 편의 기능을 완벽히 연동하였습니다.

3. 개발 과정의 한계점 및 트러블슈팅(Troubleshooting)

프로젝트 진행 과정에서 소프트웨어와 하드웨어를 결합하는 단계 중 기구학적 한계에 직면하였으며, 전체 시스템의 동작 안정성을 확보하기 위해 다음과 같은 구조적 타협점 및 개선을 도출하였습니다.

하중 초과로 인한 기구부 축소 및 모터 고정:

설계 초기 구상했던 4관절 로봇 암 구조에서 램프 헤드부의 하중이 예상치를 초과함에 따라, 중

력 방향의 부하를 직접적으로 받는 2번, 3번 관절 모터에 지속적인 과부하 및 하드웨어 에러 상태가 발생하였습니다. 이를 해결하기 위해 무리한 모터 구동으로 인한 전체 보드 셧다운을 방지하고자 2, 3번 관절을 물리적으로 고정하고 램프의 지지대(기둥) 길이를 축소하여 하중 토크를 최소화하는 안정화 작업을 최우선으로 진행하였습니다. 현재는 팬/틸트(좌우, 상하) 역할을 하는 1번, 4번 모터만을 활용하여 안정적인 트래킹을 수행하도록 로직을 수정하였습니다.

구조 변경으로 인한 비전 AI 환경(FOV)의 제약:

안정성을 위해 램프의 전체적인 높이가 낮아짐에 따라, 당초 구현을 완료하였던 사물 인식 AI(booktest.py)의 카메라 시야각(FOV) 확보가 불가능해졌습니다. 책상의 넓은 면적을 비추지 못하여 '학습용 책 감지' 기능을 부득이하게 최종 시연 기능에서 제외하였습니다. 하지만 해당 모델의 Keras 학습 파이프라인과 추론 로직은 성공적으로 구축되어 있으므로, 추후 하드웨어 프레임이 개선되면 즉각적으로 도입할 수 있는 상태입니다.

4. 총평 및 향후 발전 방향

본 프로젝트인 플로우램프는 비록 하드웨어 하중 설계 측면에서 일부 아쉬움이 있었으나, 임베디드 보드(라즈베리파이) 환경 내에서 멀티스레딩 및 서브프로세스를 활용한 무거운 AI 모델의 비동기 처리, 스마트폰 앱과 하드웨어 간의 실시간 API 통신, 수학적 렌더링에 기반한 동적 하드웨어 제어라는 컴퓨터공학적 핵심 과제들을 매우 높은 수준으로 통합해 냈습니다.

향후 3D 프린팅 소재를 경량화하거나 고토크 BLDC 모터를 채용하여 기구부의 하드웨어 스펙을 보강한다면, 본 프로젝트에서 완성해 둔 Keras 기반 사물 인식 로직과 고도화된 다관절 제어 알고리즘을 온전히 발휘할 수 있을 것입니다. 이번 캡스톤디자인은 단순한 소프트웨어 개발을 넘어 물리적 하드웨어와 상호작용하는 융합 시스템 설계의 복잡성과 해결 과정을 깊이 있게 경험한 성공적인 프로젝트로 평가할 수 있습니다.

팀원별 역할 및 수행 업무

권태정(조장)

하드웨어(3D 모델링) 설계 및 구조 최적화

플로우램프 전체 외형 및 내부 기구부 3D 모델링 설계 주도

공학적 구조 개선: 초기 설계 시 기둥 길이가 길어 모터에 과도한 하중(토크 부하)이 걸리는 문제 발견 -> 기존 3개/3개 기둥 구조를 1개/0개 구조로 과감히 변경하여 하드웨어 경량화 및 구동 안정성 확보

컴퓨터 비전(MediaPipe) 기반 핵심 기능 선행 개발 및 PoC(개념 검증)

핸드 트래킹: 초반부 MediaPipe 기반 21개 손가락 랜드마크 추출 로직 구현 및 'V'자 제스처 인식 테스트 완료

거북목 감지: 초기 거북목 판정 알고리즘 설계 및 조건 충족 시 화면 내 'TURTLENECK' 경고 메시지 출력 기능 검증

졸음 감지: 눈 깜빡임 분석 코드 개발, 5초 이상 폐안(눈 감음) 지속 시 '자고 있음' 상태 판정 및 메시지 출력 스트레스 테스트 완료

책 인식 : 초기 책 인식 알고리즘 설계 및 분석 코드 개발

사용자 경험(UI/UX) 중심의 전용 앱 프로토타이핑

초기 UI 설계: 비전 인식 결과(자세 점수, 졸음 상태 등)와 램프 제어 기능을 직관적으로 시각화할 수 있는 모바일 전용 어플리케이션 UI/UX 프로토타입 디자인 제작 담당

시스템의 핵심 기능과 앱 인터페이스 간의 연동 흐름을 사전 정의하여 전체 소프트웨어 개발 방향성 제시

프로젝트 관리(PM) 및 행정 지원매주 진행되는 정기 회의록 작성 및 주간 진도보고서 등 기술 문서 자산화 전담최종 발표를 위한 PPT 기획 및 시각화 자료 제작 총괄

서성준 :

비전 알고리즘 고도화 및 모터 · LED 하드웨어 연동

핸드 트래킹 및 모터 제어: 초기 손바닥 인식 단계를 넘어서, 실시간 손바닥 중심 좌표를 산출하고 이를 구동 모터 제어 알고리즘과 연동하여 카메라 추적(Tracking) 시스템 완성

제스처 기반 제어: 손목의 회전 각도 및 모션을 분석하여 플로우램프의 LED 밝기를 정밀 조절하는 인터랙션 기능 구현

핵심 비즈니스 로직 개발 및 데이터 관리(DB)

거북목 감지 및 점수화: 초기 거북목 판별 코드를 기반으로, 사용자의 집중도를 정량화하는 '학습 점수' 및 '자세 점수' 산출 알고리즘 수립

데이터베이스 구축: 산출된 실시간 자세 · 학습 데이터를 DB에 저장하고 체계적으로 관리하는 파이프라인 구축 완료

졸음 감지: 초기 졸음 감지 코드의 안정성 향상 및 예외 처리 보조 참여

책 인식 : 책 인식 알고리즘 설계 및 분석 코드 개발

하드웨어 구조 최적화 (공동 수행)

플로우램프 장축 기동으로 인한 모터 토크 부하 문제를 해결하기 위해, 기존 3개/3개 기동 구조를 1개/0개 구조로 축소하는 외형 경량화 및 하드웨어 수정 작업 공동 수행

프로젝트 PM 보조 및 진척도 관리

전체 개발 파트의 상세 세부 일정 및 기술별 구현 진척도(WBS)를 상시 측정하고 리스크를 관리하여 프로젝트 완수에 기여

최영인

모바일 어플리케이션(UI/UX) 및 기능 개발 총괄

서비스 목적에 맞춘 전용 모바일 앱의 전체 UI 인터페이스 디자인 및 프론트엔드 기능 구현 주도

사용자가 실시간 데이터(자세 점수, 학습 점수 등)를 직관적으로 확인할 수 있도록 모바일 대시보드 및 컨트롤 화면 설계

하드웨어 초기 조립 및 기반 구축

프로젝트 초반부 3D 출력물 및 각종 전자 부품(Raspberry Pi, 모터, LED 등)의 하드웨어 전체 조립 및 물리적 인터페이스 구조 안정화 수행

임베디드-앱 연동 및 시스템 통신 제어

모터 제어 연동: 모바일 앱과 하드웨어를 연동하여, 사용자가 어플리케이션 내에서 구동 모터를 직접 원격 제어할 수 있는 시스템 구현

LED 및 통신 보조: 앱-램프 간 통신 프로토콜 정립을 보조하고, 어플리케이션을 통한 LED 밝기 및 모드 제어 기능 연동 지원

이준영

앱-하드웨어 네트워크 통신 메인 개발

모바일 어플리케이션과 임베디드 시스템 간의 실시간 데이터 교환을 위한 통신 프로토콜 설계 및 메인 개발 완수

앱과 디바이스 간의 안정적인 데이터 파이프라인을 구축하여 지연 없는 양방향 제어 환경 확보

멀티 카메라 시스템 구축 및 하드웨어 연동

듀얼 카메라 비전 환경 구현: 시스템 몸통(Body) 카메라와 램프 헤드(Head) 카메라의 다중 비전 입력을 안정적으로 캡처하고 시스템에 연동하는 작업 전담

PWM 기반 LED 제어: LED PWM(주파수 변조) 하드웨어 회로 신호선을 연결하고, 모바일 앱에서 하드웨어 LED 신호를 직접 정밀 제어할 수 있도록 펌웨어 로직 구현

소프트웨어 아키텍처 통합 및 코드 리팩토링 (Main 파일 구축)

파트별로 분산되어 있던 독립적 모듈(비전, 모터, LED 등)의 스파게티 코드를 main.py 중심의 모듈화 구조로 전면 리팩토링 및 통합 관리

단일 진입점(Entry Point) 구성을 통해 코드 호출 구조를 단순화하고 시스템 구동 최적화 달성

하드웨어 빌드 및 DevOps/유지보수 총괄

형상 관리 인프라 구축: 프로젝트 초기 전용 Git 리포지토리 개설 및 개발 브랜치 전략 수립을 통한 협업 환경 인프라 조성

프로젝트 초반부 3D 프린팅 파트 및 주요 전자 부품의 전체 하드웨어 조립 참여

개발 전 과정에서 발생하는 병목 현상 해결 및 전체 소프트웨어 스택의 버그 디버깅/유지보수 총괄

VI. 기타

1. 개발 및 테스트 환경

운영체제(OS): 원활한 컴퓨터 비전 처리와 AI 모델 구동을 위해 메인 개발 환경으로 Ubuntu Linux를 사용하며, 하드웨어 제어 테스트를 병행함.

사용 언어 및 프레임워크: 영상 처리 및 제어 로직 구현을 위해 Python을 주력 언어로 사용하며, OpenCV 및 MediaPipe 라이브러리를 적극 활용함.

하드웨어 제어: 실시간 영상 데이터 처리와 다이내믹셀 모터의 3축 역운동학(IK) 제어를 연동하기 위해 라즈베리파이(Raspberry Pi)간의 시리얼 통신을 구축함.

2. 예산 및 조달 제약

총 예산 80만 원 내에서 로봇 구동의 핵심인 고가의 다이내믹셀 모터 및 제어 보드 비용을 우선적으로 충당함. 이를 위해 램프의 외형 프레임, 관절 브라켓 등은 교내 3D 프린터를 활용하여 PLA 소재로 자체 제작함으로써 구조적 안정성을 챙기면서도 제작 단가를 대폭 절감함.

[참고문헌]

[1] Google. "MediaPipe Hand Landmarker," Internet: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker. [2026년 3월 19일].

[2] OpenCV. "OpenCV: Open Source Computer Vision Library," Internet: <https://opencv.org>. [2026년 3월 19일].

[3] J. J. Craig. "Introduction to Robotics: Mechanics and Control," 4th ed., Pearson, 2017.

[4] R. Szeliski. "Computer Vision: Algorithms and Applications," 2nd ed., Springer, 2022.

[5] 박준형, 최영진. "MediaPipe와 역운동학을 이용한 로봇 팔 원격 제어 인터페이스 구현," 한국로봇학회 논문지, Vol. 18, 210-218, 2023.

VII. 부록

핵심 주요 코드

3D 벡터 내적을 활용한 손가락 관절 각도 계산 – handtest.py

```
def joint_angle_3d(p1, vertex, p2):
    """
    세 개의 3D 랜드마크 포인트를 이용해 관절의 각도를 계산하는 함수.
    벡터의 내적(Dot Product)과 아크코사인(Arccosine)을 활용하여 사이 각도를 도출.
    """
    # 두 개의 3D 방향 벡터 생성
    v1 = (p1.x - vertex.x, p1.y - vertex.y, p1.z - vertex.z)
    v2 = (p2.x - vertex.x, p2.y - vertex.y, p2.z - vertex.z)

    # 벡터의 크기(Norm) 계산
    norm1 = math.sqrt(sum(value * value for value in v1))
    norm2 = math.sqrt(sum(value * value for value in v2))
    if norm1 == 0 or norm2 == 0:
        return 0.0
    # 코사인 제2법칙을 활용한 두 벡터 사이의 코사인 값 도출 및 각도 변환
    cosine = sum(a * b for a, b in zip(v1, v2)) / (norm1 * norm2)
    return math.degrees(math.acos(clamp(cosine)))

def is_finger_extended(lm, mcp_idx, pip_idx, dip_idx, tip_idx):
    """
    계산된 관절 각도와 팁(Tip) 길이를 종합하여 특정 손가락이 펴진 상태인지 판별.
    """
    pip_angle = joint_angle_3d(lm[mcp_idx], lm[pip_idx], lm[dip_idx])
    dip_angle = joint_angle_3d(lm[pip_idx], lm[dip_idx], lm[tip_idx])
    tip_distance = dist_3d(lm[0], lm[tip_idx])
    pip_distance = dist_3d(lm[0], lm[pip_idx])
    # 관절이 일정 각도(FINGER_STRAIGHT_ANGLE) 이상 펴져있고, 끝마디 거리가 충분한지
    확인
    return (
        pip_angle >= FINGER_STRAIGHT_ANGLE
        and dip_angle >= FINGER_STRAIGHT_ANGLE
        and tip_distance >= pip_distance * FINGER_EXTENSION_RATIO
    )
```


화면 오차 기반 다이나믹셀 모터 트래킹 제어 – handtest.py

```
def error_to_axis(error, dead_zone, max_error):  
    """  
    손바닥의 중심과 화면 중앙 사이의 오차(error)를 모터 제어용 축 벡터(-1.0 ~ 1.0)로 변환.  
    미세한 떨림 방지를 위해 데드존(Dead Zone)을 적용.  
    """  
    # 오차가 데드존 이내이면 모터를 움직이지 않음  
    if abs(error) <= dead_zone:  
        return 0.0  
    usable_range = max(max_error - dead_zone, 1)  
    direction = 1.0 if error > 0 else -1.0  
    magnitude = (abs(error) - dead_zone) / usable_range  
    return clamp(direction * magnitude)  
  
def smooth_axis(previous, target):  
    """  
    모터의 급격한 움직임과 기계적 무리를 방지하기 위한 스무딩(지수 가중 이동 평균) 적용.  
    """  
    alpha = clamp(MOTOR_AXIS_SMOOTHING, 0.0, 1.0)  
    next_value = previous + (target - previous) * alpha  
    max_step = max(0.0, MOTOR_AXIS_MAX_STEP)  
    # 한 번에 변화할 수 있는 최대 스텝(max_step)을 제한하여 부드러운 움직임 구현  
    if max_step and abs(next_value - previous) > max_step:  
        next_value = previous + max_step * (1.0 if next_value > previous else -1.0)  
    if target == 0.0 and abs(next_value) < MOTOR_AXIS_STOP_EPSILON:  
        return 0.0  
    return clamp(next_value)
```

아크탄젠트(atan2)를 이용한 주먹 회전각 및 조명 밝기 제어 - handtest.py

```
def get_hand_rotation(hand_landmarks):
    """
    손의 랜드마크 중 검지 밑동(index_base)과 새끼손가락 밑동(pinky_base)의
    좌표 차이를 이용해 손 전체의 회전 각도를 구하는 함수.
    """
    lm = hand_landmarks.landmark
    index_base = lm[5]
    pinky_base = lm[17]
    dx = pinky_base.x - index_base.x
    dy = pinky_base.y - index_base.y
    # 아크탄젠트(atan2)를 활용하여 x, y 변화량을 각도(Degree)로 변환
    angle = math.degrees(math.atan2(dy, dx))
    # -90 ~ 90도 범위로 정규화
    if angle > 90:
        angle -= 180
    elif angle < -90:
        angle += 180
    return angle
# --- 메인 루프 적용 부분 ---
# 기준 회전각(fist_rotation_reference) 대비 현재 회전각의 변화량(delta_angle)을 계산
delta_angle = smooth_rotation - fist_rotation_reference
# 우회전 시: 임계값(ROTATION_BRIGHTNESS_THRESHOLD)을 넘으면 밝기 증가 API 호출
if delta_angle > ROTATION_BRIGHTNESS_THRESHOLD:
    delta = ROTATION_BRIGHTNESS_STEP
    if BRIGHTNESS_LEVEL < BRIGHTNESS_MAX:
        api_brightness = send_brightness_gesture("brightness_up")

# 좌회전 시: 밝기 감소 API 호출
elif delta_angle < -ROTATION_BRIGHTNESS_THRESHOLD:
    delta = ROTATION_BRIGHTNESS_STEP
    if BRIGHTNESS_LEVEL > BRIGHTNESS_MIN:
        api_brightness = send_brightness_gesture("brightness_down")
```

라즈베리파이 환경 맞춤형 카메라 파이프라인 최적화 (MJPEG 파싱) - facetest.py

```
class RpicamVideoCapture:
```

```
    """
```

라즈베리파이 카메라 모듈의 프레임 저하 및 지연(Latency) 문제를 해결하기 위해,
OS 명령어(rpicam-vid)의 표준 출력(stdout) 스트림에서 직접 이미지를 읽어오는 커스텀 클래스.

```
    """
```

```
    def __init__(self, camera_index, width, height, fps):
```

```
        # ... (초기화 및 subprocess.Popen 실행 부분 생략) ...
```

```
        pass
```

```
    def read(self):
```

```
        # 서브프로세스의 스트림이 열려있지 않으면 종료
```

```
        if not self.isOpened():
```

```
            return False, None
```

```
        while True:
```

```
            # 버퍼에서 4096 바이트씩 청크(Chunk) 단위로 읽어옴
```

```
            chunk = self.stdout.read(4096)
```

```
            if not chunk:
```

```
                return False, None
```

```
            self.buffer += chunk
```

```
            # JPEG 이미지의 시작(FF D8)과 끝(FF D9) 바이너리 시그니처를 탐색
```

```
            start = self.buffer.find(b"\xff\xd8")
```

```
            end = self.buffer.find(b"\xff\xd9", start + 2)
```

```
            # 불완전한 프레임인 경우 다음 스트림을 계속 읽음
```

```
            if start == -1:
```

```
                self.buffer = self.buffer[-1:]
```

```
                continue
```

```
            if end == -1:
```

```
                self.buffer = self.buffer[start:]
```

```
                continue
```

```
            # 완전한 1프레임(jpg)이 확보되면 버퍼에서 분리
```

```
            jpg = self.buffer[start:end + 2]
```

```
            self.buffer = self.buffer[end + 2:]
```

```
            # 바이트 배열을 OpenCV가 처리할 수 있는 numpy 배열 이미지로 디코딩 (IMREAD_COLOR)
```

```
            frame = cv2.imdecode(np.frombuffer(jpg, dtype=np.uint8), cv2.IMREAD_COLOR)
```

```
            if frame is not None:
```

```
                return True, frame
```

양어깨와 귀의 중점을 이용한 거북목(Turtle Neck) 각도 계산 – facetest.py

```
def midpoint(p1, p2):
    """
    두 3D 랜드마크 포인트(예: 좌/우 어깨, 좌/우 귀)의 중앙 좌표(x, y)를 계산하여 반환.
    """
    class Point:
        pass
    p = Point()
    p.x = (p1.x + p2.x) / 2
    p.y = (p1.y + p2.y) / 2
    return p

def calculate_neck_angle(shoulder_center, ear_center):
    """
    어깨 중점과 귀 중점을 연결한 선분과 수직선 사이의 각도를 도출하여 거북목 정도를 측정.
    """
    # x축의 변화량(dx)과 y축의 변화량(dy) 계산 (화면상 y좌표는 아래로 갈수록 커지므로 어깨-귀 순서로 뺌)
    dx = ear_center.x - shoulder_center.x
    dy = shoulder_center.y - ear_center.y

    # 아크탄젠트(atan2)를 사용하여 라디안을 구하고, 이를 degree 단위(각도)로 변환
    # 목이 앞으로 기울어질수록(dx가 커질수록) 각도가 증가함
    angle = math.degrees(math.atan2(abs(dx), abs(dy)))
    return angle
```

학습 품질 정량화 (자세 및 집중도 점수 환산 로직) - facetest.py

```
def calculate_scores(total_study_time, good_posture_time, away_time, drowsy_time,
turtle_neck_count, drowsy_count):
    """
    측정된 누적 시간과 감지 횟수를 바탕으로 자세 점수(50점)와 집중 점수(50점)를 계산하여
    총점 도출.
    """
    if total_study_time <= 0:
        return 0, 0, 0

    # [1] 자세 점수 (50점 만점)
    # 총 학습 시간 중 바른 자세(good_posture_time)를 유지한 비율로 기본 점수 산정
    posture_ratio = good_posture_time / total_study_time
    posture_score = int(posture_ratio * 50)

    # 거북목이 감지된 횟수 1회당 1점씩 페널티 부여
    posture_score -= turtle_neck_count * 1
    # 점수가 0~50점 범위를 벗어나지 않도록 클램핑(Clamping)
    posture_score = max(0, min(50, posture_score))

    # [2] 집중 점수 (50점 만점)
    # 총 시간에서 자리 이탈 시간과 졸음 시간을 제외한 '순수 학습 시간(pure_study_time)'
    # 도출
    pure_study_time = max(0, total_study_time - away_time - drowsy_time)
    focus_ratio = pure_study_time / total_study_time
    focus_score = int(focus_ratio * 50)

    # 자리 이탈이 누적 5분(300초) 이상일 경우 추가 페널티 5점 부여
    if away_time >= 300:
        focus_score -= 5
    focus_score = max(0, min(50, focus_score))

    # 총점 합산 (100점 만점)
    total_score = posture_score + focus_score

    return posture_score, focus_score, total_score
```

일일 학습 데이터 누적 및 SQLite 데이터베이스 갱신 - facetest.py

```
def save_results_to_db(result_data, db_path=DB_PATH):
    """
    현재 세션의 학습 결과를 SQLite DB에 저장.
    동일한 날짜(study_date)의 기록이 이미 존재하면 데이터를 덮어쓰지 않고 기존 값에 '누적
    연산'을 수행.
    """
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    # DB에 오늘 날짜의 학습 기록이 이미 존재하는지 조회
    cursor.execute("SELECT ... FROM study_records WHERE study_date = ?",
(result_data["study_date"],))
    existing = cursor.fetchone()

    if existing:
        # 기존 데이터가 존재하면 각 항목(시간, 횟수 등)에 현재 세션의 데이터를 더하여 누
        적
        turtle_neck_count = old_turtle_neck_count + result_data["turtle_neck_count"]
        # ... (중략: 다른 변수들도 동일하게 합산) ...

        # 누적된 전체 시간과 횟수를 바탕으로 최종 점수(Score)를 새롭게 재계산
        posture_score, focus_score, total_score = calculate_scores(...)

        # 합산된 데이터와 재계산된 점수로 기존 레코드 UPDATE
        cursor.execute("UPDATE study_records SET turtle_neck_count = ?, ... WHERE
study_date = ?", (...))
    else:
        # 기존 데이터가 없으면 새로운 날짜의 레코드로 최초 INSERT
        cursor.execute("INSERT INTO study_records (...) VALUES (...)", (...))

    conn.commit()
    conn.close()
```

Keras 모델 로딩 및 DepthwiseConv2D 호환성 문제 해결 - booktest.py

```
class BookClassifier:
    def __init__(self, model_path=MODEL_PATH, label_path=LABEL_PATH,
confidence_threshold=CONFIDENCE_THRESHOLD):
        import tf_keras

        # 사전 학습된 모델(특히 MobileNet 기반 모델) 로드 시 발생할 수 있는
        # DepthwiseConv2D 레이어의 'groups' 파라미터 호환성 에러를 방지하기 위한 커스
        # 톼 클래스
        class CompatibleDepthwiseConv2D(tf_keras.layers.DepthwiseConv2D):
            def __init__(self, *args, groups=None, **kwargs):
                super().__init__(*args, **kwargs)

        print("Book AI model loading...")

        # custom_objects 매개변수를 활용하여 수정된 호환성 클래스를 주입하며 모델 로드
        self.model = tf_keras.models.load_model(
            model_path,
            compile=False,
            custom_objects={
                "DepthwiseConv2D": CompatibleDepthwiseConv2D,
            },
        )
        # 텍스트 파일에서 라벨(클래스 이름)을 읽어오고 불필요한 숫자/공백 전처리
        self.labels = [clean_label(label) for label in load_labels(label_path)]
        self.confidence_threshold = confidence_threshold
```

딥러닝 입력을 위한 이미지 정규화(Normalization) 및 전처리 - booktest.py

```
def preprocess_image(frame):
    """
    OpenCV 영상 프레임을 딥러닝 모델의 입력 규격에 맞게 전처리하는 함수.
    """
    # 1. 모델의 입력 사이즈(224x224)에 맞게 이미지 크기 조절
    resized = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))

    # 2. 연산을 위해 데이터 타입을 부동소수점(float32)으로 변환
    image = resized.astype(np.float32)

    # 3. 픽셀 값(0~255)을 모델 학습에 최적화된 범위인 -1.0 ~ 1.0으로 정규화
    (Normalization)
    image = (image / 127.5) - 1.0

    # 4. Keras 모델은 배치(Batch) 단위 입력을 요구하므로, 차원을 하나 추가 (예: (224, 224, 3) -> (1, 224, 224, 3))
    image = np.expand_dims(image, axis=0)

    return image
```

관심 영역(ROI) 추출을 통한 연산량 최적화 - booktest.py

```
def classify(self, frame):
    h, w, _ = frame.shape

    # 화면의 가로, 세로 중 더 작은 값이나 300px 중 최소값을 취해 정사각형 Box 사이즈 결정
    box_size = min(300, h, w)

    # 화면 정중앙을 기준으로 관심 영역(ROI: Region of Interest)의 좌상단, 우하단 좌표 계산
    x1 = w // 2 - box_size // 2
    y1 = h // 2 - box_size // 2
    x2 = x1 + box_size
    y2 = y1 + box_size

    # 전체 프레임에서 중앙의 책이 있을 만한 영역만 슬라이싱(Slicing)하여 추출
    roi = frame[y1:y2, x1:x2]

    # 추출된 ROI 영역만 딥러닝 모델에 입력하여 추론 속도 및 정확도 향상
    input_data = preprocess_image(roi)
    prediction = self.model.predict(input_data, verbose=0)[0]
```


다이나믹셀 레지스터 기반 속도 제어 모드(Velocity Mode) 초기화 - moter.py

```
ADDR_OPERATING_MODE = 11      # 동작 모드를 설정하는 주소
ADDR_TORQUE_ENABLE = 64       # 모터의 토크(힘)를 켜고 끄는 주소
ADDR_GOAL_VELOCITY = 104      # 목표 속도값을 입력하는 주소
MODE_VELOCITY_CONTROL = 1     # 속도 제어 모드 (Velocity Control Mode) 값

def connect(self):
    """
    통신 포트를 열고 모터를 속도 제어 모드로 초기화하는 함수.
    다수의 스레드에서 동시 접근하는 것을 막기 위해 Lock(동기화)을 사용함.
    """
    # ... (포트 및 통신속도 설정 부분 생략) ...

    with self._lock: # 스레드 충돌 방지용 락 활성화
        self.connected = True
        self._disabled_motors.clear()

        # 설정된 모든 모터(1번 좌우, 4번 상하)에 대해 초기화 수행
        for motor_id in self.config.motor_ids:
            try:
                # 1. 안전한 설정을 위해 먼저 모터의 토크를 끄 (TORQUE_OFF)
                self._write_1_byte(motor_id, self.ADDR_TORQUE_ENABLE,
self.TORQUE_OFF)

                # 2. 동작 모드를 '속도 제어 모드(Velocity Control)'로 변경하여 부드러운
                #   트래킹 준비
                self._write_1_byte(
                    motor_id,
                    self.ADDR_OPERATING_MODE,
                    self.MODE_VELOCITY_CONTROL,
                )

                # 3. 초기 목표 속도를 0으로 설정하여 급발진 방지
                self._write_4_bytes(motor_id, self.ADDR_GOAL_VELOCITY, 0)

                # 4. 설정 완료 후 다시 토크를 켜서 구동 준비 완료 (TORQUE_ON)
                self._write_1_byte(motor_id, self.ADDR_TORQUE_ENABLE,
self.TORQUE_ON)
            except RuntimeError as exc:
                self._disable_motor(motor_id, str(exc))
```

비전 오차를 모터의 2축(X, Y) 속도로 변환하는 매핑 로직 - moter.py

```
def move_xy(self, x: float, y: float, speed: int = 2):
    """
    비전 시스템에서 산출된 정규화된 X, Y 오차(-1.0 ~ 1.0)를 모터별 속도로 매핑.
    상하/좌우 모터의 무게 및 부하 차이를 고려하여 Vertical Multiplier를 적용.
    """
    # 최고 제한 속도(max_velocity)를 넘지 않도록 속도 값 클램핑
    speed = max(0, min(int(speed), self.config.max_velocity))

    # 램프 헤드의 무게를 버텨야 하는 상하(Y축) 모터는 가중치(Multiplier)를 곱해 더 큰
    힘으로 보정
    vertical_speed = max(
        0,
        min(
            self.config.max_velocity,
            round(speed * self.config.vertical_velocity_multiplier),
        ),
    )

    # 데드존 및 정규화 처리 (-1.0 ~ 1.0)
    x = self._normalize_axis(x)
    y = self._normalize_axis(y)

    # 1번 모터(팬, 좌우)와 4번 모터(틸트, 상하)에 각각 목표 속도 할당
    # 카메라 상하 반전 등을 고려하여 y축은 음수(-)로 방향 전환
    velocities = {
        1: self._axis_to_velocity(x, speed),
        4: self._axis_to_velocity(-y, vertical_speed),
    }
    return self.set_goal_velocities(velocities)
```

기구부 파손 방지를 위한 소프트 리미트(Soft Limit) 제어 - moter.py

```
def _limit_velocity_by_position(self, motor_id: int, velocity: int):  
    """  
    모터의 현재 위치(Position)를 읽어와 물리적인 동작 한계치(Soft Limit)에 도달했는지  
확인.  
    기구부가 충돌하거나 선이 꼬이는 것을 방지하기 위해 한계 지점에서는 속도를 0으로  
차단.  
    """  
    if not self.config.enable_soft_limits or velocity == 0:  
        return velocity  
  
    # 모터의 현재 절대 위치(엔코더 값)를 읽어옴  
    position = self._read_position(motor_id)  
    margin = max(0, self.config.soft_limit_margin)  
  
    # 음수(-) 방향으로 회전 중일 때, 설정된 최소 각도(min_position)에 도달하면 정지  
    if velocity < 0 and position <= self.config.min_position + margin:  
        return 0  
  
    # 양수(+) 방향으로 회전 중일 때, 설정된 최대 각도(max_position)에 도달하면 정지  
    if velocity > 0 and position >= self.config.max_position - margin:  
        return 0  
  
    return velocity # 안전 범위 내에 있으면 원래 속도 반환
```

하드웨어 에러 디코딩 및 시뮬레이션 폴백(Fallback) - moter.py

@staticmethod

```
def _decode_hardware_error_status(status: int):
```

```
    """
```

모터 컨트롤러에서 반환한 1바이트의 에러 상태 코드(Hardware Error Status)를 비트마스킹(Bitmasking)하여 구체적인 에러 원인으로 디코딩함.

```
    """
```

다이나믹셀 데이터시트 기반의 비트별 에러 원인 매핑

```
bits = (
```

```
    (0, "input voltage"),    # 0번 비트: 입력 전압 이상
```

```
    (2, "overheating"),      # 2번 비트: 모터 과열
```

```
    (3, "motor encoder"),    # 3번 비트: 엔코더(위치 센서) 오류
```

```
    (4, "electrical shock"), # 4번 비트: 전기적 충격
```

```
    (5, "overload"),        # 5번 비트: 과부하 (힘이 부족할 때)
```

```
)
```

켜져 있는(1) 비트들만 추출하여 문자열 리스트로 반환

```
return [name for bit, name in bits if status & (1 << bit)]
```

```
def _handle_connection_error(self, message: str):
```

```
    """
```

모터 포트 연결 실패나 치명적 에러 발생 시 처리.

강제 종료(Crash)를 방지하기 위해 가상으로 동작하는 Simulation 모드로 전환.

```
    """
```

```
if not self.simulate_on_error:
```

```
    raise RuntimeError(message)
```

```
print(f"{message}; 시뮬레이션 모드로 전환합니다.")
```

```
self.connected = False
```

```
self.simulation = True # 하드웨어 없이 소프트웨어적으로만 작동 상태 시뮬레이션
```

```
return self.status()
```

스레드(Thread)를 활용한 비동기(Asynchronous) 거북목 경고등 제어 - led.py

```
def start_posture_alert(self):
    """
    거북목 경고 발생 시, 메인 API 스레드의 응답을 지연시키지 않고
    독립적인 백그라운드 스레드에서 LED를 깜빡이도록 처리하는 비동기 함수.
    """
    # 이미 경고등이 작동 중이면 중복 실행 방지
    if self.posture_alert_running:
        return

    self.posture_alert_running = True
    self._posture_alert_stop.clear() # 중지 이벤트 플래그 초기화

    def run_blink():
        # stop 이벤트가 외부에서 호출되기 전까지 무한 반복 (빨간색 <-> 꺼짐)
        while not self._posture_alert_stop.is_set():
            show_alert_color((255, 0, 0))
            # time.sleep 대신 wait를 사용하여 대기 중에도 즉시 스레드 종료가 가능하
            # 도록 구현
            if self._posture_alert_stop.wait(0.35):
                break
            show_alert_color((0, 0, 0))
            if self._posture_alert_stop.wait(0.35):
                break

        # 경고가 종료되면 기존에 사용자가 설정해둔 색상으로 자동 복구
        self._apply_color(*self.current_color)
        self.posture_alert_running = False

    # 데몬 스레드(daemon=True)로 생성하여 메인 프로그램 종료 시 안전하게 함께 종료
    # 되도록 설정
    self._posture_alert_thread = threading.Thread(
        target=run_blink,
        name="led-posture-alert",
        daemon=True,
    )
    self._posture_alert_thread.start()
```

하드웨어 스펙을 고려한 색상 순서 보정 및 밝기 스케일링 - led.py

```
COLOR_ORDER = "GRB" # WS2812B LED의 하드웨어 색상 배열 순서
def _scale(self, value):
    """
    입력된 색상 값(0~255)에 현재 시스템의 전체 밝기(brightness) 비율을 적용하여 스케일링.
    """
    # (현재 색상값 * 밝기설정값) / 255 공식을 통해 실제 출력할 최종 픽셀 밝기 도출
    return max(0, min(255, value * self.brightness // 255))
def _order_color(self, red, green, blue):
    """
    사용자 친화적인 RGB 입력을 하드웨어 요구 스펙인 GRB 순서로 변환하고 밝기를 적용. """
    values = {
        "R": self._scale(red),
        "G": self._scale(green),
        "B": self._scale(blue),
    }
    # COLOR_ORDER 변수("GRB")에 맞춰 재배열된 리스트 반환
    return [values[channel] for channel in self.COLOR_ORDER]
```

스레드 안전성(Thread-Safety)을 보장하는 하드웨어 제어 방어 로직 - led.py

```
def _apply_color(self, r, g, b):
    """최종적으로 LED 스트립 전체에 색상을 입히는 내부 함수.
    여러 스레드에서 동시에 SPI 하드웨어 버스에 접근하는 것을 막기 위해 Lock 사용. """
    # 전원이 꺼진 상태라면 강제로 모든 색상을 0(블랙)으로 처리
    if not self.is_on:
        r, g, b = 0, 0, 0
    # 비정상적인 색상 값이 입력되어도 시스템이 다운되지 않도록 0~255 사이로 클램핑
    (Clamping)
    r = self._clamp_color_value(r)
    g = self._clamp_color_value(g)
    b = self._clamp_color_value(b)
    # 다중 스레드 환경에서 SPI 버스 충돌을 방지하기 위한 뮤텍스(Mutex) 락 획득
    with self._lock:
        self._fill((r, g, b))

    if self.simulation:
        state = "ON" if self.is_on else "OFF"
        print(f"[LED {state}] R:{r} G:{g} B:{b} brightness:{self.brightness_percent}%")
```

비전 AI 처리를 위한 서브프로세스(Subprocess) 격리 및 관리 – main.py

```
class VisionScriptThread:
```

```
    """
```

```
    무거운 딥러닝/비전 연산이 메인 API 서버의 응답 속도에 영향을 주지 않도록,  
    독립된 서브프로세스(Subprocess)로 분리하여 실행하고 수명 주기를 관리하는 래퍼 클래스.  
    """
```

```
    def __init__(self, name: str, script_path: Path, extra_env: dict[str, str] | None = None,  
python_executable: str | None = None):
```

```
        # ... (초기화 생략) ...
```

```
        self.process: subprocess.Popen | None = None
```

```
        # 프로세스 상태를 모니터링하기 위한 별도의 데몬 스레드 생성
```

```
        self.thread = threading.Thread(target=self._run, name=f"vision-{name}",  
daemon=True)
```

```
    def _run(self):
```

```
        env = os.environ.copy()
```

```
        env.update(self.extra_env) # 하위 프로세스에 API URL 등 환경변수 주입
```

```
        try:
```

```
            # os.system이 아닌 Popen을 사용하여 프로세스의 입출력 및 종료 상태를 비동  
            기적으로 제어
```

```
            self.process = subprocess.Popen(  
                [self.python_executable, str(self.script_path)],  
                cwd=PROJECT_ROOT,  
                env=env,  
            )
```

```
            return_code = self.process.wait() # 프로세스가 종료될 때까지 대기
```

```
        except OSError as exc:
```

```
            print(f"Vision script failed to start: {self.name}: {exc}")
```

```
            return
```

Thread와 Asyncio를 결합한 하드웨어 런타임 제어 – main.py

```
class LampRuntime:
```

```
    """
```

램프의 다양한 상태(대기 모드, 일반 모드 등)를 비동기(Asyncio) 기반으로 제어하는 런타임.

메인 스레드와 분리된 전용 스레드에서 이벤트 루프를 실행함.

```
    """
```

```
    def _run_event_loop(self):
```

```
        # 램프 하드웨어 제어만을 위한 독립적인 비동기 이벤트 루프 생성
```

```
        self._loop = asyncio.new_event_loop()
```

```
        asyncio.set_event_loop(self._loop)
```

```
        self._main_task = self._loop.create_task(self.start())
```

```
        # ... (생략) ...
```

```
    def _run_threadsafe(self, coroutine_factory):
```

```
        """
```

FastAPI 서버(다른 스레드)에서 하드웨어 모드를 변경하려 할 때,

충돌 없이 비동기 루프에 작업을 예약(Scheduling)하는 동기화 브릿지 함수.

```
        """
```

```
        if not self._ready.wait(timeout=5):
```

```
            raise RuntimeError("Lamp runtime thread is not ready.")
```

```
        # run_coroutine_threadsafe를 사용하여 다른 스레드에서 안전하게 코루틴 실행
```

```
        future = asyncio.run_coroutine_threadsafe(coroutine_factory(), self._loop)
```

```
        return future.result(timeout=5)
```